

OTTO-FRIEDRICH UNIVERSITY OF BAMBERG

CHAIR OF EXPLAINABLE MACHINE LEARNING

Deep Learning Exercise

Assignment 4



Term: WS 24/25

Instructor: Prof. Dr. Christian Ledig

Teaching Assistants: Sebastian Dörriech,
Jonas Alle

Start: Monday, 06th of January 2025, 08:00am

Deadline: Sunday, 02nd of February 2025, 11:59pm

Discussions Theory: [VC-Forum](#)

Discussions Programming: [VC-Forum](#)

Guidelines

- We generally encourage you to collaborate with other students. You may talk to a friend or create a post in VC to discuss the questions and potential directions for solving them. However, this is not a group assignment. Hence, each student must write his/her/their own answers and explicitly mention any collaborators.
- We will apply anti-cheating software to check the submissions for plagiarism. Besides all submissions of the current semester, we further check for online resources as well as submissions of previous semesters. Anyone who is flagged by the software will automatically receive 0 points for the homework. If you feel that you were accused wrongly, please approach your TA immediately.
- Do NOT change the filenames and function definitions in the skeleton code provided. This will cause the test scripts to fail and results in you receiving 0 points for the respective task. You may also NOT change the import modules in each file or import additional modules.
- It is your responsibility to make sure that all code and other deliverables are in the correct format and that your submission compiles and runs. Failing to do so will result in the reduction of points.
- It is your responsibility to hand in the submission in time. Anyone failing to do so will automatically receive 0 points for the homework. If you were not able to submit your work due to illness or other reasons, please approach your TA immediately to clarify further action.
- We hope you enjoy your dive into the area of Deep Learning with its broad range of applications. Deep Learning is considered the driver of AI with its limitations having yet to be explored. Please have fun coding and solving the challenging questions, so you don't remain the dinosaur, Mark Cuban labeled you as back in 2017.



Related Classes and Resources

- [Deep Learning, Georgia Institute of Technology](#)
- [Deep Learning, Eberhard Karls Universität Tübingen](#)
- [Deep Learning, FAU Erlangen-Nuremberg](#)
- [Deep Learning Specialization, DeepLearning.AI](#)
- [Deep Neural Networks with PyTorch, IBM](#)
- [Convolutional Neural Networks for Visual Recognition, Stanford](#)
- [Machine Learning, Oxford](#)
- [Deep Learning, New York University](#)
- [Deep Learning, CMU](#)
- [Deep Learning, University of Maryland](#)
- [Hugo Larochelle's Neural Networks class](#)



Necessary Tools and Those That Make Your Life Easier

PyCharm - The IDE for Python Programmers

If you have never coded in **Python** before, and even if you have, we recommend you to check out [PyCharm](#) (by [JetBrains](#)), a powerful IDE for programming in Python. [PyCharm](#) comes in a free community version as well as in billed professional one. The community version already includes all the features you might need. However, the professional version includes some additional ones which could come in handy. Therefore, you might want to check out the latter. The great deal about using software from JetBrains is that they offer [free educational licenses](#) for their products. Hence, as a student you can apply for such a licence and use all their billed products for free during your time of study.

[PyCharm](#) offers a bunch of features like allowing to look for the occurrences of a keyword anywhere in a project or allowing to split the tool window in order to maintain a good overview of the code. It also comes with a version control system and an in-built debugger to avoid the need for countless print statements.

Jupyter Notebooks

Next to regular Python files, you will also be using [Jupyter Notebooks](#) throughout this class. Formerly known as IPython Notebooks, [Jupyter Notebooks](#) are an interactive computational environment, in which you can combine code execution, rich text, mathematics, plots and media. The combination of markdown and code cells allow such Notebooks to provide a clear guidance through entire coding projects while enabling the visualization of data and results next to their associated code. This allows for doing data science with Python for that IDEs like [PyCharm](#) are not ment for. For more details on [Jupyter Notebook](#), please see the official [Jupyter website](#) or this [beginner's guide](#) hosted on medium.

DataSpell - The IDE for Data Scientists

For all of you who like writing code in a robust IDE like [PyCharm](#) which offers features such as autocompletion, spellchecker, version control, easy terminal access, database integrations, or a debugger, coding in [Jupyter Notebook](#) (which in fact lacks all of those) might seem like a nuisance to you. Hence, you might want to check out [DataSpell](#) (by JetBrains) and IDE for data scientists. In contrast to [PyCharm](#), unfortunately [DataSpell](#) does not come in a free community version but only in a payable one. However, since it is a product of JetBrains you can use it for free with your already obtained [free educational student license](#).

[DataSpell](#) offers great coding assistance (smart-code completion, error checking, etc.) and markdown support ("notebook style"). It has the ability to output visualizations without the need for extra installations and works well with interactive visualization libraries. It comes with a version control, terminal, database tools as well as a debugger and offers also the use of plugins.

Google Colab

[Google Colab](#) is a free cloud service based on [Jupyter Notebooks](#) that supports the usage of free GPU resources provided by Google. This service enables anybody to develop deep learning applications without the need for purchasing expensive hardware. It has some limits with respect to the your session or file sizes but in this class we won't be using it to such an extent that you will have to worry about those. You can create notebooks in Colab, upload notebooks, store notebooks, share notebooks, and lots of other things. The only requirement is that you will need a free [Google account](#) which can be created easily in a couple of minutes since [Google Colab](#) is directly connected to and makes use of [Google Drive](#). If you have never worked with [Google Colab](#), please check out the following tutorials by [Towards Data Science](#) or [GeeksforGeeks](#) to understand how to set up a project in this framework.

We will be using [Google Colab](#) later in this class. But we still recommend to check it out as soon as possible and get familiar with its processes, so that you don't have to worry about it mid-semester when your assignments might start to pile up.

Initializing Your Tools

Python and Dependencies

In this assignment, we will work with **Python 3**. If you have never used Python before, please make yourself familiar with the language first by studying one of its numerous, free tutorials, e.g. the [Official Python Tutorial](#) or the [W3 Schools Python Tutorial](#). If you do not have a python distribution installed yet, we recommend installing [Miniconda](#) (or [Anaconda](#)) with Python 3. Refer to the [users' manual](#) for more details about the use of conda. We will provide you with the file `environment.yaml` which contains the libraries needed for this assignment. You can further use it to create an assignment-specific conda environment which will help you get rid off any potential import or package version errors. To do so, open the [Anaconda Prompt](#) within the folder for this assignment and run the following command:

```
$ conda env create -f environment.yaml
```

It is important to note that [PyTorch](#) which we will be using for training and testing of our networks is currently not working for Python versions > 3.9. To avoid package dependency issues when creating your conda environment we have specified the use of Python 3.9 explicitly in `environment.yaml`.

If you already have your own Python development environment, please refer to the environment file to find the necessary libraries for this assignment.

PyCharm

To create a [PyCharm](#) project you have to execute the following steps:

1. If you have never opened [PyCharm](#) you will be forwarded to a "Welcome screen" after running the application for your first time. Click **New Project**.
If you've already worked with [PyCharm](#) and you run the application, you will always be forwarded to your last project page. Here you have to select **File** → **New Project** from the main menu.
2. To initialize the already created conda environment as your interpreter you have to select **Pure Python** → **Previously configured interpreter**. If your conda environment is not shown in the dropdown menu, you have to select ... → **Conda Environment** →
Interpreter: C:\...\.conda\envs\xai-dl-a4\python.exe and click **Ok**
3. Choose the project location. Enter the path to your local **Assignment 4** copy within **Location:** to specify the directory for your project and hit **Create** afterwards.
4. You will get a notification saying **Directory is Not Empty - Do you want to create project from existing sources?**. This is fine, you can simply click **Create from Existing Sources** and start your project in [PyCharm](#).

Jupyter

If you used the `environment.yaml` file to create your development environment for this assignment, you have already installed **Jupyter Notebook** and you don't have to do anything else.

If you used your own environment and don't have **Jupyter Notebook** yet, install it using one of the following commands:

```
$ conda install -c anaconda jupyter
$ pip install jupyter
```

After installing **Jupyter Notebook** successfully, to run it in your browser, first you have to activate your conda environment within the **Anaconda Prompt**:

```
$ conda activate xai-dl-a4
```

This ensures that you run your **Jupyter Notebook** with the Python version and packages we need for the assignment. This helps to avoid import and package errors. Afterwards navigate to the directory of your local **Assignment 4** copy. Here you have to enter

```
$ jupyter notebook
```

within the **Anaconda Prompt** to start **Jupyter Notebook** automatically in your browser. **Jupyter Notebook** will always open within the folder you have specified before by navigating to the desired directory.

DataSpell

To create a **DataSpell** project you have to execute the following steps:

1. After starting the application you have to select **File** → **Open** from the main menu and specify the path to your local **Assignment 4** copy.
2. You have to specify the interpreter for your **DataSpell** project. Hence, you have to select **<No Interpreter>** from the bottom-right and specify your conda environment. If your conda environment is not within the dropdown menu of **Python Interpreter**:, you have to click **Add Interpreter** → **Add Local Interpreter...** and specify the path to your **xai-dl-a4** environment.

Google Colab

The most important commands are:

1. Mount your drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

-
2. Test whether your GPU is running:

```
!nvidia-smi
```

3. Run external Python files with command line parameters:

```
!python "/content/drive/.../file.py" --para1 "..."
```

Submission Instructions

Besides checking your submissions by hand, we will be using automated tests, too. Please read the following instructions for submitting your solutions carefully, since failure to follow these instructions may result in parts of your assignment not being graded. We will not entertain regrading requests for failing to follow the instructions.

All exercises or questions marked with **[Bonus]** do not have to be done to achieve the maximum amount of points for their respective assignment. However, these tasks can be used to replace another task of the same assignment or guarantee that you achieve the maximum amount of points possible. All bonus tasks can only be used for their respective assignment and can not be used to improve the results in a later assignment. This means one can at max get 100 Points for each assignment.

Theory

For the theory part of the homework you have to upload one single PDF containing the answers to all the questions. Please note that the solution to each problem/sub-problem must be on a separate page. This will help us to grade your submissions in less time. Please use the provided template to create your submission.

Please also state any collaborators you have exchanged your thoughts and ideas with under the section **Colaborators**. If you have worked completely on your own just enter **None** in that section.

After finishing the theory part, generate the resulting PDF and submit it via VC.

However, if you are tasked with running experiments in the programming section of this assignment, you will need to create the resulting report, add it at the end of your theory submission PDF to create a single PDF and submit this in the end via VC.

For the report you will use the provided template to collect and summarize your experimental results and findings. Simply go through all of the provided questions within the template to do so. For whichever questions asking for plots, please include plots from all your experiments.

Explanations should go into why things work the way they do with proper deep learning theory. For example, with hyperparameter tuning you should explain the reasoning behind your choices and what behavior you expected. If you need more than one slide for a question, you are free to create new slides **right after the given one**.

In the end you will export your report in PDF format and combine it with your submission of the theory questions to a single PDF before submitting the resulting PDF via VC.

Coding

For your convenience and to assure that your code can be processed by our test cases you are only allowed to write code between the `# YOUR CODE HERE` comments:

```
#####
#                               START OF YOUR CODE                               #
# TODO:                                                                  #
#     ...                                                                  #
#####
.
.
.
#####
#                               END OF YOUR CODE                               #
#####
```

For Jupyter Notebooks or within DataSpell, after writing your code, you can run the cell by either pressing SHIFT+ENTER or by clicking on "Run Cell" (denoted by a play symbol) in the upper bar of the notebook.

It is important to note that after you modified the code within a cell you always have to run that cell again before the changes are recognized

After passing all provided test, make sure you don't forget to upload your submission to VC. For that you will need to submit a zip file containing all your codes. For your convenience, we provide a handy script for you that does that directly. Use the "collect_submission"-script, which checks all files for completeness and zips the files you need for the upload.

Try running:

```
$ bash collect_submission.sh
```

or if you are using a Windows machine, try running:

```
$ C:\...\assignmentfolder> collect_submission.bat
```

Before submitting your assignment, please make sure you are not doing the following:

1. You have not added any *extra* print statement(s) in the assignment.
2. You have not added any *extra* code cell(s) in the assignment.
3. You have not changed any of the function parameters.
4. You are not using any global variables inside your graded exercises. Unless specifically instructed to do so, please refrain from it and use the local variables instead.
5. You are not changing the assignment code where it is not required, like creating extra variables.

If you do any of the mentioned, our test scripts will fail and as a result you will receive **0 points** for the respective task.

Theory [50 points + 5 points Bonus]

Please use the provided template **writeup.tex** to collect your solutions to the theory questions below. Please note that the solution to each problem/subproblem must be on a separate page. Furthermore, please list any collaborators you have exchanged your thoughts and ideas with. After finishing this part, generate the resulting PDF and submit it via VC.

1 Automatic Differentiation [18 points]

In practice, writing the closed-form expression of the derivative of a loss function f w.r.t. the parameters of a deep neural network is hard (and mostly unnecessary) as f becomes complex. Instead, we define computation graphs and use the automatic differentiation algorithms (typically backpropagation) to compute gradients using the chain rule.

For example, consider the expression:

$$f(x, y) = (x + y)(y + 1) \quad (1)$$

Let's define intermediate variables a and b such that:

$$\begin{aligned} a &= x + y \\ b &= y + 1 \\ f &= a \times b \end{aligned} \quad (2)$$

A computation graph for the “forward pass” through f is shown in Figure 1.

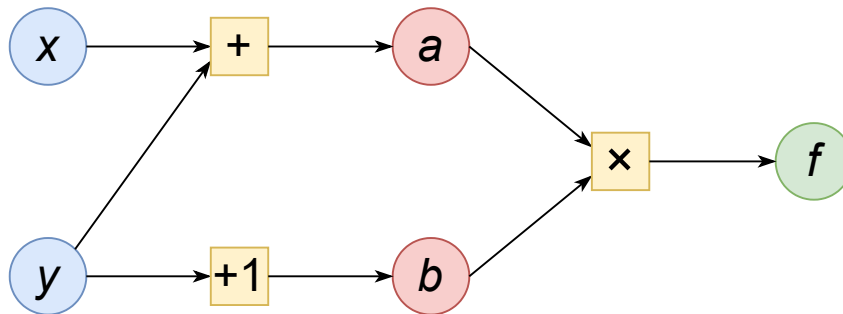


Figure 1: Computation Graph for the “forward pass” through f .

We can then work backwards and compute the derivative of f w.r.t. each intermediate variable $\left(\frac{\partial f}{\partial a}, \frac{\partial f}{\partial b}\right)$ and chain them together to get $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$.

Remark:

Does this ring a bell? We have done something similar in **Assignment 1**, when we were talking about **Directed Acyclic Graphs**. However, this time we will project the idea onto neural networks and their underlying graph structure. Moreover, we will introduce you to one particular task the machine is doing for you over and over again, so you don't have to get your hands dirty. Namely, automatic differentiation. Maybe after this exercise you will value your laptop differently than before and actually have fallen in love with automatic differentiation ;)

1.1 Computation Graph [5 points]

Let $\sigma(\cdot)$ denote the standard sigmoid function. Now, for the following vector function:

$$\mathbf{f} = \begin{pmatrix} f_1(w_1, w_2) \\ f_2(w_1, w_2) \end{pmatrix} = \begin{pmatrix} \cos(w_1) \cos(w_2) + \sigma(w_2) \\ \ln(w_1 + w_2) + w_1^2 w_2 \end{pmatrix} \quad (3)$$

- a) Draw the computation graph.
- b) Compute the value of f at $\mathbf{w} = (1, 2)$.

1.2 Numerical Differentiation [3 points]

At this $\mathbf{w}$, compute the Jacobian $\frac{\partial \mathbf{f}}{\partial \mathbf{w}}$ using numerical differentiation (using $\Delta w = 0.01$).

1.3 Forward Mode Auto-Differentiation [5 points]

At this $\mathbf{w}$, compute the Jacobian using forward mode auto-differentiation.

1.4 Backward Mode Auto-Differentiation [4 points]

At this $\mathbf{w}$, compute the Jacobian using backward mode auto-differentiation.

1.5 So, what do you think? [1 point]

Don't you love that software exists to do this for us ;)?

2 Convolution Basics [10 points]

The convolution layer inside of a CNN is intrinsically an *affine transformation*. A vector is received as input and is multiplied with a matrix to produce an output (to which a bias vector is usually added before passing the result through a nonlinearity). This operation can be represented as $y = Ax$, in which A describes the affine transformation.

2.1 Convolution as Matrix Operation [2 points]

We will first revisit the convolution layer as discussed in the class. Consider a convolution layer with a 3×3 kernel W , operated on a single input channel X , represented as:

$$W = \begin{bmatrix} w_{(0,0)} & w_{(0,1)} & w_{(0,2)} \\ w_{(1,0)} & w_{(1,1)} & w_{(1,2)} \\ w_{(2,0)} & w_{(2,1)} & w_{(2,2)} \end{bmatrix} \quad \text{and} \quad X = \begin{bmatrix} x_{(0,0)} & x_{(0,1)} & x_{(0,2)} \\ x_{(1,0)} & x_{(1,1)} & x_{(1,2)} \\ x_{(2,0)} & x_{(2,1)} & x_{(2,2)} \end{bmatrix} \quad (4)$$

Now let us work out a **stride-4** convolution layer, with **zero padding size of 2**. Consider ‘flattening’ the input tensor X in row-major order as:

$$X = [x_{(0,0)}, x_{(0,1)}, x_{(0,2)}, \dots, x_{(2,0)}, x_{(2,1)}, x_{(2,2)}]^T \quad (5)$$

Write down the convolution as a matrix operation A such that: $Y = AX$. Output Y is also flattened in row-major order.

2.2 Transposed Convolution as affine Transformation [2 points]

Recall that transposed convolution can help us upsample the feature size spatially. Consider a transposed convolution layer with a 2×2 kernel W operated on a single input channel X , represented as:

$$W = \begin{bmatrix} w_{(0,0)} & w_{(0,1)} \\ w_{(1,0)} & w_{(1,1)} \end{bmatrix} \quad \text{and} \quad X = \begin{bmatrix} x_{(0,0)} & x_{(0,1)} \\ x_{(1,0)} & x_{(1,1)} \end{bmatrix} \quad (6)$$

We ‘flattened’ the input tensor in row-major order as $X = [x_{(0,0)}, x_{(0,1)}, x_{(1,0)}, x_{(1,1)}]^T$.

Write down the affine transformation A corresponding to a transposed convolution layer with kernel W , **stride 2, no padding**. Output Y is also flattened in row-major order.

2.3 Convolution $\Longleftrightarrow$ Transposed Convolution [6 points]

Convolution layers in most CNNs consist of multiple input and output feature maps. The collection of kernels form a 4D tensor (output channels o , input channels i , filter rows k , filter columns k), represented in short as (o, i, k, k) . For each output channel, each input channel is convolved with a distinct 2D slice of the 4D kernel and the resulting set of feature maps is summed element-wise to produce the corresponding output feature map.

There is an interesting property that a convolutional layer with kernel size $(o \times r^2, i, k, k)$ is identical to a transposed convolution layer with kernel size $(o, i, k \times r, k \times r)$. Here the word ‘identical’ means with the same input feature X , both operations will give the same output Y with only a difference in the ordering of flattened elements of Y .

Now let us prove the property in a restricted setting. Consider $o = 1, r = 2, i = 1, k = 1$. Given the same input feature X as in equation 6, write down the affine transformation for a convolutional layer with kernel size $(4, 1, 1, 1)$, and show that it is an operation identical to a transposed convolution layer with kernel size $(1, 1, 2, 2)$. Use whatever parameters for stride and padding that make the output Y the same size (size meaning number of elements, not the same shape). Y should have 16 elements.

3 Logic and XOR [4 points]

3.1 AND and OR [2 points]

Implement AND and OR for pairs of binary inputs using a single linear threshold neuron with weights $\mathbf{w} \in \mathbb{R}^2$, bias $b \in \mathbb{R}$, and $x \in \{0, 1\}^2$:

$$\begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0 \end{cases} \quad (7)$$

That is, find w_{AND} and b_{AND} such that:

x_1	x_2	$f_{\text{AND}}(\mathbf{x})$
0	0	0
0	1	0
1	0	0
1	1	1

Table 1: Truth table for AND

and find w_{OR} and b_{OR} such that:

x_1	x_2	$f_{\text{OR}}(\mathbf{x})$
0	0	0
0	1	1
1	0	1
1	1	1

Table 2: Truth table for OR

3.2 XOR [2 points]

Consider the XOR function:

x_1	x_2	$f_{\text{XOR}}(\mathbf{x})$
0	0	0
0	1	1
1	0	1
1	1	0

Table 3: Truth table for XOR

Prove that XOR can NOT be represented using a linear model with the same form as equation 7.

Hint:

To see why, plot the examples from above in a plane and think about drawing a linear boundary that separates them.

4 Piece-wise Linearity [5 points]

Consider a specific 2 hidden layer ReLU network with inputs $x \in \mathbb{R}$, 1 dimensional outputs, and 2 neurons per hidden layer. This function is given by:

$$h(x) = W^{(3)} \max\{0, W^{(2)} \max\{0, W^{(1)}x + b^{(1)}\} + b^{(2)}\} + b^{(3)} \quad (8)$$

with weights:

$$W^{(1)} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \quad W^{(3)} = \begin{bmatrix} 1 & 1 \end{bmatrix} \quad (9)$$

and bias:

$$b^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad b^{(3)} = -1 \quad (10)$$

An interesting property of networks with piece-wise linear activations like the ReLU is that on the whole they compute piece-wise linear functions.

For $x = x_o = 2$, determine the value of weight $W \in \mathbb{R}$ and bias $b \in \mathbb{R}$ such that:

$$\left. \frac{dh(x)}{dx} \right|_{x=x_o} = W \quad \text{and} \quad Wx_o + b = h(x_o) \quad (11)$$

5 Depth - Composing Linear Pieces [11 points]

Now we'll turn to a more recent result that highlights the **Deep** in Deep Learning. Depth (composing more functions) results in a favorable combinatorial explosion in the “number of things that a neural net can represent”. For example, to classify a cat it seems useful to first find parts of a cat: eyes, ears, tail, fur, etc. The function which computes a probability of cat presence should be a function of these components because this allows everything you learn about eyes to generalize to all instances of eyes instead of just a single instance. Below you will detail one formalizable sense of this combinatorial explosion for a particular class of piecewise linear networks.

Consider:

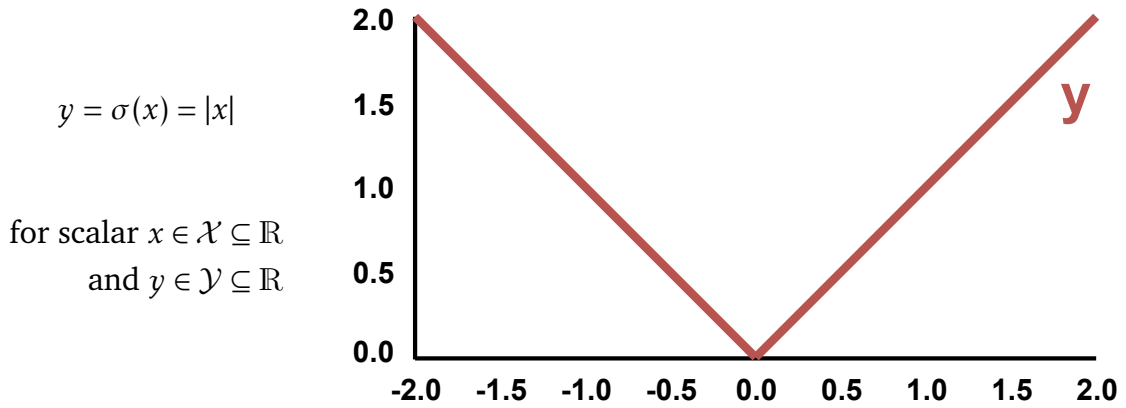


Figure 2: $|x|$

The function has one linear region on $x < 0$ and another on $x > 0$ and the activation identifies these regions, mapping both of them to $y > 0$. More precisely, for each linear region of the input, $\sigma(\cdot)$ is a bijection. There is a mapping to and from the output space and the corresponding input space. However, given an output y , it's impossible to tell which linear region of the input it came from, thus $\sigma(\cdot)$ identifies (maps on top of each other) the two linear regions of its input. This is the crucial definition because when a function identifies multiple regions of its domain that means any subsequent computation applies to all of those regions. When these regions come from an input space like the space of images, functions which identify many regions where different images might fall (e.g., slightly different images of a cat) automatically transfer what they learn about a particular cat to cats in the other regions.

More formally, we will say that $\sigma(\cdot)$ identifies a set of M disjoint input regions $\mathcal{R} = \{R_1, \dots, R_M\}$ (e.g. $\mathcal{R} = \{(-1, 0), (0, 1)\}$) with $R_i \subseteq \mathcal{X}$ onto one output region $O \subseteq \mathcal{Y}$ (e.g. $(0, 1)$) if for all $R_i \in \mathcal{R}$ there is a bijection from R_i to O .

Recall that a bijection from X to Y is a function $\mu : X \rightarrow Y$ such that for all $y \in Y$ there exists a **unique** $x \in X$ with $\mu(x) = y$.

5 DEPTH - COMPOSING LINEAR PIECES ~~Number of~~ Identified Regions [4 points]

Start by applying the above notion of identified regions to linear regions of one layer of a particular neural net that uses absolute value functions as activations. Let $x \in \mathbb{R}^d$ and $y \in \mathbb{R}^d$ (outputs are in some feature space, not a label space. Normally a linear classifier would be placed on top of what we are here calling y). And pick weights $W^{(1)} \in \mathbb{R}^{d \times d}$ and bias $b^{(1)} \in \mathbb{R}^d$ as follows:

$$W_{ij}^{(1)} = \begin{cases} 2 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad \text{and} \quad b_i^{(1)} = -1 \quad (12)$$

Then one layer of a neural net with absolute value activation functions is given by:

$$f_1(x) = |W^{(1)}x + b| \quad (13)$$

Note that this is an absolute value function applied piece-wise and not a norm.

5.1 Number of Identified Regions [4 points]

How many regions of the input are identified onto $O = (0, 1)^d$ by $f_1(\cdot)$? Prove it.

5.2 Composition of two bijection functions [2 points]

Next consider what happens when two of these functions are composed. Suppose g identifies n_g regions of $(0, 1)^d$ onto $(0, 1)^d$ and f identifies n_f regions of $(0, 1)^d$ onto $(0, 1)^d$. How many regions of its input does $f \circ g(\cdot)$ identify onto $(0, 1)^d$?

5.3 Multiple Layers [5 points]

Finally consider a series of L layers identical to the one presented in section 5.

$$\begin{aligned} h_1(x) &= |W_1 x + b_1| \\ h_2(x) &= |W_2 h_1 + b_2| \\ &\vdots \\ h_L(x) &= |W_L h_{L-1} + b_L| \end{aligned} \quad (14)$$

Let $x \in (0, 1)^d$ and $f(x) = h_L$. Note that each h_i is implicitly a function of x . Show that $f(x)$ identifies 2^{Ld} regions of its input.

5.4 Conclusion

Now compare the number of identified regions for an L layer net to that of an $L - 1$ layer net. The L layer net can separate its input space into 2^d more linear regions than the $L - 1$ layer net. On the other hand, the number of parameters and the amount of computation time grows linearly in the number of layers. In this very particular sense (which does not always align well with practice) deeper is better.

To summarize this problem set, you have shown a number of results about the representation power of different neural net architectures:

1. Neural nets (even single neurons) can represent logical operations.
2. Neural nets we use today compute piece-wise linear functions of their input. Third, the representation power of neural nets increases exponentially with the number of layers.
3. The representation power of neural nets increases exponentially with the number of layers.

The point of the exercise was to convey intuition that removes some of the magic from neural nets representations. Specifically, neural nets can decompose problems logically, and piece-wise linear functions can be surprisingly powerful.

6 Bonus: Transformer [5 points]

Consider a sequence of 2 tokens as input to a transformer consisting of 1 self-attention layer and 1 attention head. Each token x_i is represented by a 3-dimensional vector. The token vectors $\{x_1, x_2\}$ are stacked together in rows to get $\mathbf{L}_l$. The key, query and value embeddings are 4-dimensional each. The weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$ and $\mathbf{W}^V$ for query, key and value respectively, and the output projection matrix $\mathbf{W}^L$ are provided below. Compute the query, key and value embeddings, self-attention matrix and the updated embedding vectors for each token.

Notation:

All the matrices are represented in Einstein notation. $\mathbf{L}_l$ is represented as $\mathbf{L}_l[T, J]$ where $T = 2$ is the sequence length and $J = 3$ is the dimension of the token vector. The weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$ and $\mathbf{W}^V$ are represented as $\mathbf{W}[K, I, J]$ where $K = 1$ is the number of heads and $I = 4$ is the dimension of query, key or value. Each value of the matrix is represented by $\mathbf{W}[k, i, j]$ where k, i, j iterate over K, I, J respectively. The output projection matrix $\mathbf{W}^L$ is represented as $\mathbf{W}^L[J, I]$. We use 0-indexing here.

$$x_1 = [0, 1, 1]^\top \quad x_2 = [1, 0, 0]^\top \quad \mathbf{L}_l = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \end{pmatrix} \quad \mathbf{W}^L[J, I] = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

$$\mathbf{W}^Q[0, I, J] = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & -1 \\ -1 & 1 & 1 \\ -1 & 0 & 0 \end{pmatrix} \quad \mathbf{W}^K[0, I, J] = \begin{pmatrix} -1 & 1 & 0 \\ 1 & 1 & -1 \\ -1 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \quad \mathbf{W}^V[0, I, J] = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

There are two ways to solve this task.

First, you can use the following equations for a multi-headed self-attention. For each head k and position t , we compute a **key**, **query** and **value** vector. The queries $\mathbf{Q}$ and the keys $\mathbf{K}$ are used to compute the self-attention matrix $\mathbf{A}$ for head k . $\mathbf{A}$ is then multiplied with the values $\mathbf{V}$ to yield embedding vectors $\mathbf{H}$ that are concatenated:

$$\begin{aligned} \mathbf{Q}_{l+1}[k, t, i] &= \mathbf{W}_{l+1}^Q[k, i, J] \mathbf{L}_l[t, J] \\ \mathbf{K}_{l+1}[k, t, i] &= \mathbf{W}_{l+1}^K[k, i, J] \mathbf{L}_l[t, J] \\ \mathbf{V}_{l+1}[k, t, i] &= \mathbf{W}_{l+1}^V[k, i, J] \mathbf{L}_l[t, J] \end{aligned}$$

$$\mathbf{A}_{l+1}[k, t_1, t_2] = \text{softmax}_{t_2} \left[\frac{1}{\sqrt{D_Q}} \mathbf{Q}_{l+1}[k, t_1, I] \mathbf{K}_{l+1}[k, t_2, I] \right] \quad (15)$$

$$\begin{aligned} \mathbf{H}_{l+1}[k, t, i] &= \mathbf{A}_{l+1}[k, t, T] \mathbf{V}_{l+1}[k, T, i] \\ \mathbf{L}_{l+1}[t, j] &= \mathbf{W}_{l+1}^L[j, I] (\mathbf{H}_{l+1}[1, t, I], \dots, \mathbf{H}_{l+1}[K, t, I]) \end{aligned}$$

Or you can use direct matrix notation of the above (faster ;)). It's up to you!

Code [50 points]

Overview

The programming part of this assignment consists of the following individual components. First, you will complete a simple Convolutional Neural Network (CNN) architecture from scratch and learn how to implement CNNs with PyTorch, one of the most commonly used deep learning frameworks. Second you will learn about regularization by implementing L2 regularization as well as dropout for an existing model.

Testing

- For all [Jupyter Notebooks](#), we have implemented test cells directly beneath the code snippets you are going to implement:

```
!python ./tests/<test_file> --test_case <name_of_test>
```

You can just run these test cells to test your implementations.

- Besides that many IDEs (e.g., [PyCharm](#)) offer the option to run unit tests directly.
- Additionally, you can run via the command line by using:

```
$ python -m unittest tests.<name_of_tests>
```

However, passing all local tests neither means your code is free of bugs nor guarantees that you will receive full credits for the coding section. Your code will be tested via additional tests which are not present in your local unit tests.

ToDo:

1. Implement all required files in [01_cnn](#).
2. Implement the notebook [regularization.ipynb](#) in [02_regularization](#).

7 Convolutional Neural Network [42 points]

Again, within this part, you will complete a simple Convolutional Neural Networks (CNN) architecture from scratch and learn how to implement CNNs with PyTorch, one of the most commonly used deep learning frameworks. CNNs are one of the major advancements in computer vision over the past decade. You will also run different experiments on imbalanced datasets to evaluate your model and techniques to deal with imbalanced data.

ToDo:

1. Implement all required files in `01_cnn`.

7.1 Implementing CNN from Scratch [27 points]

You will work in `./01_scratch` for this part of the assignment.

7.1.1 Data Loading [1 point]

You might have noticed that the training process on a huge amount of data can be extremely slow. Therefore, for this assignment we only want to deliberately overfit the model with a small portion of data to verify whether the model is learning something or not. Hence as a first step, you should download the dataset by running:

```
$ cd data
$ sh get_data.sh
$ cd ../
```

Microsoft Windows 10 Only:

```
$ C:\..\assignmentfolder> cd data
$ C:\..\assignmentfolder\data> get_data.bat
$ C:\..\assignmentfolder\data> cd ..
```

7.1.2 Module Implementation [19 points]

You will now learn how to build CNN from scratch. Typically, a convolutional neural network is composed of several different modules and these modules work together to make the network effective. For each module, you will implement a forward pass (computing forwarding results) and a backward pass (computing gradients). Therefore, your tasks are as follows:

- a) Follow the instructions in the code to complete each module in `./modules`. Specifically, modules to be implemented are 2D convolution [7 points], 2D Max Pooling [7 points], ReLU [2 points], and Linear [3 points]. These will be the building blocks of the full network. The file `./modules/conv_classifier.py` ties each of the aforementioned modules together and is the subject of the next section.

7.1 Implementing CNN from scratch [27 points]

b) You can test your implementation of each module by running the respective tests:

- 2D convolution → `./tests/test_conv.py`
- 2D Max Pooling → `./tests/test_maxpool.py`
- ReLU → `./tests/test_relu.py`
- Linear → `./tests/test_linear.py`

7.1.3 Network Implementation [4 points]

After finishing each module, it's time to put things together to form a real convolutional neural network. Your task is:

- a) Follow the instructions in the code to complete a CNN network in `./modules/conv_classifier.py`. The network is constructed by a list of module definitions in order and should handle both forward and backward communication between modules.

7.1.4 Optimizer [3 points]

In the last assignment you have learned about the optimization algorithm "mini-batch gradient descent with momentum". But of course we can also use the momentum term to improve the convergence rate of the stochastic gradient descent (SGD) optimizer. Doing so, we can describe the optimization process with the following equations:

$$\begin{aligned} v_t &= \beta v_{t-1} - \eta \frac{\partial L}{\partial w} \\ w &= w + v_t \end{aligned} \tag{16}$$

where β denotes the momentum coefficient and η denotes the learning rate.

- a) Follow the instructions in the code to complete SGD with momentum in `./optimizer/sgd.py`.
- b) You can test your implementation of the sgd optimizer by running the test in `./tests/test_sgd.py`

7.1.5 Training

To train your CNN model you can then simply run:

```
$ python train.py
```

which trains using only 50 samples of the CIFAR-10 dataset. The script will make a plot on the training data only and store it within the `.01_scratch`-folder. Your final accuracy should be slightly under 0.9 with the given network.

7.2 Implementing CNN with PyTorch [15 points]

You will work in `./02_pytorch` for this part of the assignment. The `main` function in `main.py` contains the major logic of the code and you can run it by:

```
$ python main.py --config configs/<name_of_config_file>.yaml
```

7.2.1 Install Torch [0.5 points]

Please note that the provided `environment.yaml` file does not contain a [PyTorch Installation](#) for you. This is due to the fact that you may run the code on a [CPU](#) or [GPU](#), or for a different [CUDA version](#). Hence, as a first step, please refer to the [official documentation](#) of [PyTorch](#) to install the version that is suited best for you and your equipment.

7.2.2 Data Loading [0.5 points]

Again as the first step, we need to download the data. You can do so by running:

```
$ cd data
$ sh get_data.sh
$ cd ../
```

Microsoft Windows 10 Only:

```
$ C:\..\assignmentfolder> cd data
$ C:\..\assignmentfolder\data> get_data.bat
$ C:\..\assignmentfolder\data> cd ..
```

7.2.3 Training [6 points]

The first thing of working with PyTorch is to get yourself familiarized with the basic training step of PyTorch.

- a) Complete the `train` and `validate` functions in `main.py`.

7.2.4 PyTorch Model [8 points]

You will now implement some actual networks with PyTorch. We provide some starter files for you in `./models`. The models for you to implement are as follows:

- a) **Two-Layer Network [4 points]**

This is the same network you have implemented from scratch in Assignment 2. You will build the model with two fully connected layers and a sigmoid activation function in between the two layers. Please implement the model as instructed in `./models/twolayer.py`

You can test your model via the file `./tests/test_twolayer.py`.

b) Vanilla Convolutional Neural Network [4 points]

You will build the model with a *convolution layer*, a *ReLU activation*, a *max-pooling layer*, followed by a *fully connected layer for classification*.

- Your *convolution layer* should use **32 output channels**, a **kernel size of 7** with **stride 1** and **zero padding**.
- Your *max-pooling* should use a **kernel size of 2** and **stride of 2**
- The *fully connected layer* should have **10 output features**.

Please implement the model as instructed in `./models/cnn.py`.

You can test your model via the file `./tests/test_vanilla_cnn.py`.

We provide you with configuration files for both of these models. You need to train the model without modifying the configuration file.

As stated before, the `main` function in `main.py` contains the major logic of the code. You can run it via the following command:

```
$ python main.py --config configs/<name_of_config_file>.yaml
```

This will allow you to train the specified network with the specified hyperparameters (in either `./configs/config_twolayer.yaml` or `./configs/config_twolayer.yaml`). You should expect the accuracy ("Best Prec @1 Accuracy:") of Two-Layer Network and Vanilla CNN to be around 0.3 and 0.4 respectively.

The script will then automatically saves the weights of the best model at the end of training within `./checkpoints`. We will evaluate your implementation by loading your model weights and evaluating the model on CIFAR-10 test data. All in all, please make sure the checkpoints of each model are saved into `./checkpoints`.

7.3 End of Exercise

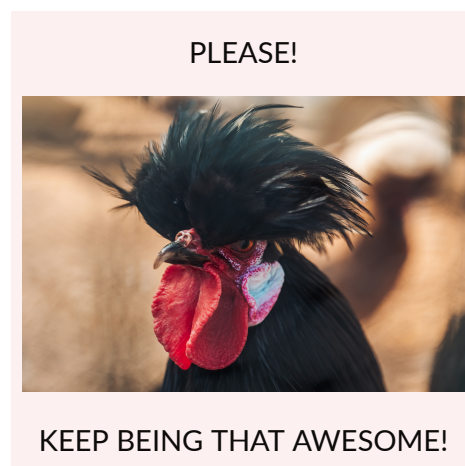


Figure 3

8 Regularization [8 points]

Deep Learning models have so much flexibility and capacity that **overfitting can be a serious problem**, if the training dataset is not big enough. Sure your model might do well on the training set, but it **doesn't generalize to new examples** that it has never seen! Regularization helps to deal with this.

ToDo:

1. Implement the notebook `regularization.ipynb` in `02_regularization`.

8.1 Visualization

These functions will be used for visualization. We provide those for you, so you can plot your results. You don't have to code anything here.

ToDo:

1. Nothing

8.2 Utils

These functions will be used as helper functions throughout the entire notebook. We provide those for you, so you can focus on the regularization task at hand. You don't have to code anything here.

ToDo:

1. Nothing

8.3 Problem Statement

You have just been hired as an AI expert by the French Football Corporation. They would like you to recommend positions where French's goal keeper should kick the ball so that the French team's players can then hit it with their head. This is illustrated in Figure 4.

Within the figure you can see a Football field. The goal keeper kicks the ball in the air and the players of each team are fighting to hit the ball with their head.

The French Football Corporation gives you the following 2D dataset from French's past 10 games.

ToDo:

1. Nothing

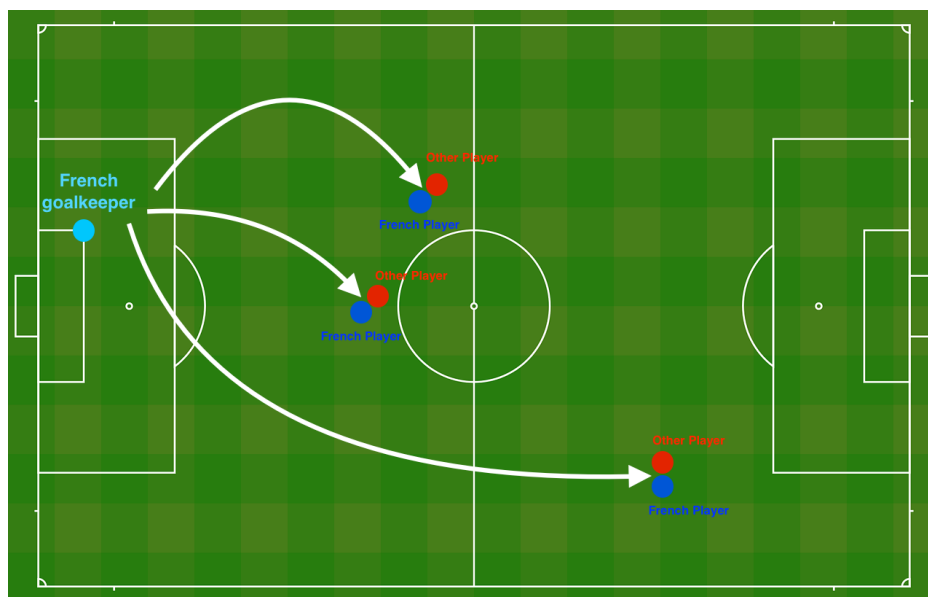


Figure 4: Visualization of the Problem Statement

8.4 Data Loading

First, we need to load our data. This step was already implemented for you below. You don't have to code anything here.

ToDo:

1. Run the **respective cell** to plot the data.

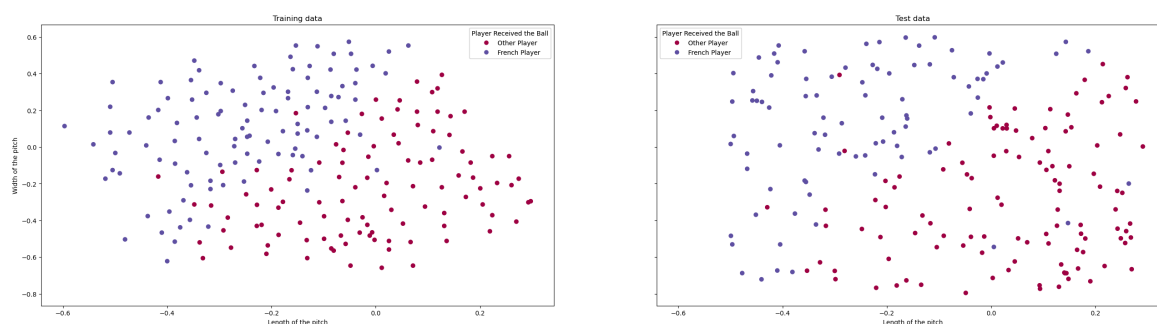


Figure 5: Dataset

Each dot corresponds to a position on the football field where a football player has hit the ball with his/her head after the French goal keeper has shot the ball from the left side of the football field.

- If the dot is blue (purple for different color scheme), it means the French player managed to hit the ball with his/her head
- If the dot is red (pink for different color scheme), it means the other team's player hit the ball with their head

Your goal is now:

To use a deep learning model to find the positions on the field where the goalkeeper should kick the ball.

First glance at the dataset:

This dataset is a little noisy, but it looks like a diagonal line separating the upper left half (blue/purple) from the lower right half (red/pink) would work well.

What you will do: You will first try a non-regularized model. Then you'll learn how to regularize it and decide which model you will choose to solve the French Football Corporation's problem.

8.5 Model

You will use the following neural network (already implemented for you below). You don't have to code anything here. This model can be used:

- In regularization mode – by setting the `lambda` input to a non-zero value. We use “`lambda`” instead of “`lambda`” because “`lambda`” is a reserved keyword in Python.
- In dropout mode – by setting the `keep_prob` to a value less than one

You will first try the model without any regularization. Then, you will implement:

- L2 regularization–functions: “`compute_cost_with_regularization()`” and “`backward_propagation_with_regularization()`”
- Dropout–functions: “`forward_propagation_with_dropout()`” and “`backward_propagation_with_dropout()`”

In each part, we will run this model with the correct inputs so that it calls the functions you've implemented. Take a look at the code below to familiarize yourself with the model.

ToDo:

1. Nothing

8.6 Non-Regularized Model

Let's train the model without any regularization, and observe the accuracy on the train/test sets.

ToDo:

1. Run the **respective cell** to train the non-regularized model variant and evaluate its results.

8.6.1 Analyzing the Non-Regularized Model

The train accuracy is **94.8%** while the test accuracy is **91.58%**. This is the baseline model (you will observe the impact of regularization on this model). Analyzing the decision boundary, we can see clearly that the non-regularized model is overfitting the training set. It is fitting the noisy points! Let's now look at two techniques to reduce overfitting.

8.7 L2 Regularization [4 points]

The standard way to avoid overfitting is called **L2 regularization**.

8.7.1 Cost Function [2 points]

L2 regularization consists of appropriately modifying your cost function, from:

$$J = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(a^{[L](i)}) + (1 - y^{(i)}) \log(1 - a^{[L](i)})) \quad (17)$$

To:

$$J_{\text{regularized}} = \underbrace{-\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(a^{[L](i)}) + (1 - y^{(i)}) \log(1 - a^{[L](i)}))}_{\text{cross-entropy cost}} + \underbrace{\frac{1}{m} \frac{\lambda}{2} \sum_l \sum_k \sum_j W_{k,j}^{[l]2}}_{\text{L2 regularization cost}} \quad (18)$$

Let's modify your cost and observe the consequences.

ToDo:

1. Implement the method `compute_cost_with_regularization` which computes the cost given by the second formula.

Hints:

1. To calculate $\sum_k \sum_j W_{k,j}^{[l]2}$, use `np.sum(np.square(Wl))`.
2. Note that you have to do this for $W^{[1]}$, $W^{[2]}$ and $W^{[3]}$, then sum the three terms and multiply by $\frac{1}{m} \frac{\lambda}{2}$.

8.7.2 Backward Propagation with Regularization [2 points]

Since we have changed the cost calculation, we have to change the backward propagation as well! All the gradients have to be computed with respect to this new cost.

ToDo:

1. Implement the method `backward_propagation_with_regularization` which applies the changes needed in backward propagation to take regularization into account.

Hints:

1. The changes only concern $dW1$, $dW2$ and $dW3$.
2. For each, you have to add the regularization term's gradient ($\frac{d}{dW}(\frac{1}{2} \frac{\lambda}{m} W^2) = \frac{\lambda}{m} W$).

8.7.3 Let's now run the model with L2 regularization

We will use $\lambda = 0.91$ for this.

ToDo:

1. Run the `respective cell` to train the model with L2 regularization and evaluate its results.

8.7.4 Analyzing L2 regularization

Congrats, the test set accuracy increased to 93%. You have saved the French football team! The training and test accuracy are both high and very similar. Therefore, you are not overfitting the training data anymore. This can also be seen when analyzing the decision boundary!

Observations:

- The value of λ is a hyperparameter that you can tune using a dev set.
- L2 regularization makes your decision boundary smoother. If λ is too large, it is also possible to “oversmooth”, resulting in a model with high bias.

What is L2-regularization actually doing?:

- L2-regularization relies on the assumption that a model with small weights is simpler than a model with large weights.
- By penalizing the square values of the weights in the cost function you drive all the weights to smaller values.
- Having large weights becomes too costly for the cost!
- This leads to a smoother model in which the output changes more slowly as the input changes.

What you should remember: the implications of L2-regularization on:

- The cost computation: A regularization term is added to the cost.
- The backpropagation function: There are extra terms in the gradients with respect to weight matrices.
- Weights end up smaller (“weight decay”): Weights are pushed to smaller values.

8.8 Dropout [4 points]

Finally, **dropout** is a widely used regularization technique that is specific to deep learning.

To understand drop-out, consider this conversation with a friend:

- **Friend:** “Why do you need all these neurons to train your network and classify images?”.
- **You:** “Because each neuron contains a weight and can learn specific features/details/shape of an image. The more neurons I have, the more features my model learns!”
- **Friend:** “I see, but are you sure that your neurons are learning different features and not all the same features?”
- **You:** “Good point... Neurons in the same layer actually don’t talk to each other. It should be definitely possible that they learn the same image features/shapes/forms/details... which would be redundant. There should be a solution.”

Dropout randomly shuts down some neurons in each iteration. Watch these two videos [./img/dropout1_kiank.mp4](#) and [./img/dropout2_kiank.mp4](#) to see what this means exactly. At each iteration, you shut down (= set to zero) each neuron of a layer with probability $1 - \text{keep_prob}$ or keep it with probability keep_prob (50% here). The dropped neurons don't contribute to the training in both the forward and backward propagations of the iteration.

When you shut some neurons down, you actually modify your model. The idea behind drop-out is that at each iteration, you train a different model that uses only a subset of your neurons. With dropout, your neurons thus become less sensitive to the activation of one other specific neuron, because that other neuron might be shut down at any time.

8.8.1 Forward Propagation with Dropout [2 points]

Implement the forward propagation with dropout. You are using a 3 layer neural network, and will add dropout to the first and second hidden layers. We will not apply dropout to the input layer or output layer.

ToDo:

1. Implement the method `forward_propagation_with_dropout` which includes dropout into the forward pass of the model.
 - a) We know that we can create a variable $d^{[1]}$ with the same shape as $a^{[1]}$ using '`np.random.rand()`' to randomly get numbers between 0 and 1. Here, you will use a vectorized implementation, so create a random matrix $D^{[1]} = [d^{1}, d^{[1](2)}, \dots, d^{[1](m)}]$ of the same dimension as $A^{[1]}$
 - b) Set each entry of $D^{[1]}$ to be 1 with probability (`keep_prob`), and 0 otherwise.
 - c) Set $A^{[1]}$ to $A^{[1]} * D^{[1]}$. (You are shutting down some neurons). You can think of $D^{[1]}$ as a mask, so that when it is multiplied with another matrix, it shuts down some of the values.
 - d) Divide $A^{[1]}$ by `keep_prob`. By doing this you are assuring that the result of the cost will still have the same expected value as without drop-out. (This technique is also called inverted dropout.)

Hints:

1. Let's say that `keep_prob=0.8`, which means that we want to keep about 80% of the neurons and drop out about 20% of them. We want to generate a vector that has 1's and 0's, where about 80% of them are 1 and about 20% are 0. This python statement: `X = (X < keep_prob).astype(int)` is conceptually the same as this if-else statement (for the simple case of a one-dimensional array):

```

for i,v in enumerate(x):
    if v < keep_prob:
        x[i] = 1
    else: # v >= keep_prob
        x[i] = 0

```

2. Note that the `x = (X < keep_prob).astype(int)` works with multi-dimensional arrays, and the resulting output preserves the dimensions of the input array.
3. Also note that without using `.astype(int)`, the result is an array of booleans `True` and `False`, which Python automatically converts to 1 and 0 if we multiply it with numbers. (However, it's better practice to convert data into the data type that we intend, so try using `.astype(int)`.)

8.8.2 Backward Propagation with Dropout [2 points]

Implement the backward propagation with dropout. As before, you are training a 3 layer network. Add dropout to the first and second hidden layers, using the masks $D^{[1]}$ and $D^{[2]}$ stored in the cache.

ToDo:

1. Implement the method `backward_propagation_with_dropout` which includes dropout into the backward pass of the model.
 - a) You had previously shut down some neurons during forward propagation, by applying a mask $D^{[1]}$ to $A1$. In backpropagation, you will have to shut down the same neurons, by reapplying the same mask $D^{[1]}$ to $dA1$.
 - During forward propagation, you had divided $A1$ by `keep_prob`. In backpropagation, you'll therefore have to divide $dA1$ by `keep_prob` again (the calculus interpretation is that if $A^{[1]}$ is scaled by `keep_prob`, then its derivative $dA^{[1]}$ is also scaled by the same `keep_prob`).

8.8.3 Let's now run the model with Dropout

We will use `keep_prob = 0.86` for this. It means at every iteration you shut down each neurons of layer 1 and 2 with 14% probability. Our function `model()` will now call:

- `forward_propagation_with_dropout` instead of `forward_propagation`.
- `backward_propagation_with_dropout` instead of `backward_propagation`.

8.8.4 Analyzing Dropout

Dropout works great! The test accuracy has increased again (to 95%)! Your model is not overfitting the training set and does a great job on the test set. The French football team will be forever grateful to you!

Note:

- A common mistake when using dropout is to use it both in training and testing. You should use dropout (randomly eliminate nodes) only in training.
- Deep learning frameworks like [PyTorch](#), [tensorflow](#), [PaddlePaddle](#), [keras](#) or [caffe](#) come with a dropout layer implementation to ease or lives.

What you should remember about dropout:

- Dropout is a regularization technique.
- You only use dropout during training. Don't use dropout (randomly eliminate nodes) during test time.
- Apply dropout both during forward and backward propagation.
- During training time, divide each dropout layer by `keep_prob` to keep the same expected value for the activations. For example, if `keep_prob` is 0.5, then we will on average shut down half the nodes, so the output will be scaled by 0.5 since only the remaining half are contributing to the solution. Dividing by 0.5 is equivalent to multiplying by 2. Hence, the output now has the same expected value. You can check that this works even when `keep_prob` is other values than 0.5.

8.9 Conclusions

Here are the results of our three models:

Model	Train Accuracy	Test Accuracy
3-layer NN without regularization	95%	91.5%
3-layer NN with L2-regularization	94%	93%
3-layer NN with dropout	93%	95%

Table 4: Comparison of regularization and non-regularization methods.

8.10 End of Exercise

UNBELIEVABLE!!!



YOU DID IT!

Figure 6